AGENT FIGHTER

Developer Report — Milestone 0 to Present

17 July 2026 · Full build audit against Build Spec v1 · 67 commits · Prepared by Claude Code

EXECUTIVE SUMMARY

Six milestones in sixty-eight hours — and one ungated assumption underneath all of them.

Agent Fighter began at commit `2e1a3d8` on 14 July 2026 at 14:06 and reached its current state on 17 July at 10:30 — **67 commits in under three days**. In that window the project shipped all six milestones its Build Spec scoped as the whole of v1: a deterministic simulation core, a full Marvel-vs-Capcom combat system, an AI sprite-generation Studio, online rollback netcode with a verifying match server, single-player AI with a 12-character roster, a credits economy with real persistence, and wagering with escrow settlement.

The build is live. The Railway match server reports `persistence:true` across 12 characters and 3 stages on protocol 3; the Vercel client returns 200. The verify gate is green: four packages typechecked, 67 core tests, 28 server tests, zero TODO markers in source.

The finding that matters is not a bug. It is that the spec's own blocking gate for Milestone 1 — "people who play MvC say it feels right" — has never been run, and five milestones are now stacked on top of it. The spec anticipated exactly this and said so in writing: *"Milestones 0–1 are the whole ballgame: if the core doesn't feel like MvC with one character, nothing downstream matters."* Everything in this report is downstream of that sentence.

68h	67	15,871	95	12	0
M0 to present	commits	lines of TS source	tests green	characters	external playtests

Source lines exclude tests (1,844) and generated assets. "Tests green" = 67 core + 28 server. Roster of 12 against a spec target of 4–6.

THE BUILD

Milestone 0 to present

The spec laid out seven milestones (0–6) and estimated engineering weight at core+combat 35%, studio+pipeline 30%, netcode+server 20%, AI+meta 15%. What follows is what actually happened, reconstructed from the commit history rather than from memory.

Phase	When	Landed	Evidence
M0 Sim skeleton	07-14 14:06	Deterministic core: fixed 60Hz step, 24.8 fixed-point math, seeded RNG, snapshot/restore, replay-hash test, local 2P demo.	`2e1a3d8`
M1 MvC feel	07-14 23:11	Full combat in one commit, 9 hours after M0: 6 buttons, magic-series chains via cancel graph, launcher + air combos, juggle points, 236/214/623 specials, projectile, super + 3-bar meter, block/chip/pushblock, throws + techs, damage scaling, hitstop, best-of-3.	`1332fca`

Phase	When	Landed	Evidence
M2 Studio MVP	07-14 23:30	Zero-dep Node server + SPA embedding the engine. Generate/Frames/Moves/Cancels/Test tabs, AI sprite pipeline (bg removal, palette lock, QC, auto-hitbox drafts), atlas packing.	`4428c27`
Guardrails	07-15 00:21	The pivotal non-feature commit. Verify gate, golden replay lock (6 pinned match hashes), mechanical determinism guards, cross-tool AGENTS.md contract, CI.	`5d70d0c`
Sprite war	07-15 00:26–09:36	14 commits fighting AI-art consistency: system animations, facing detection, median-cut palette fix, strip generation (costume drift), Gemini provider with true reference conditioning, anime art direction, parallax stages.	`dfa79d1` … `9f0c516`
M4 AI + roster	07-15 18:32	Deterministic AI as an InputSource: intent system, reaction delay, execution flubs, personality, adaptation counters, combo book from the cancel graph. One lever: skill 0–100. XP/leveling. Roster passed 4–6 and kept going.	`cefd864`
M3 Online	07-16 02:58	Trust spine per ADR 0003: matchmaking, pinned setup, WS relay with input ledger, server re-simulation naming the deviating side. GGPO rollback client-side. Then AIR Kit identity + Supabase persistence.	`d1b2e12` `9328a3d`
M5/M6 Economy	07-17 00:24	Credits per ADR 0004: mandatory sign-in, daily grant, ranked solo fees, wager pots with escrow. AIR reputation write-back. Leaderboard. Money logic mirrored in SQL + TS.	`d8c02df` `e7e6803`
Hardening	07-17 03:36–09:00	Zero-latency solo (protocol v3), P0 loop redesign (quick match, rematch, stakes card), VS screen, disconnect settlement ladder (ADR 0005), escrow sweeper, match resume, Railway + Vercel deploy.	`141b5ec` … `c940603`

Note the ordering: M4 (single-player) shipped *before* M3 (online), and the Studio landed 19 minutes after the combat system. The spec's milestone numbers were followed in spirit, not in sequence — which was the right call, since M4's AI is what later made ranked solo possible without a house-bot process.

SCORECARD

Where we stand against the spec

#	Milestone	Spec definition of done	Status
0	**Sim skeleton**	Deterministic core, snapshot/restore + replay hash green in CI, local 2P	**DONE**
1	**It feels like MvC**	Full combat system + *gate: people who play MvC say it feels right*	**CODE COMPLETE** **GATE NEVER RUN**
2	**Studio MVP**	Character #2 built entirely through Studio in under 2 days	**DONE** **~5 minutes**
3	**Online**	Rollback over WebRTC + match server + ledger + verification	**DONE** **relay, not P2P**
4	**Single-player + roster**	FSM AI at 3 difficulties; roster to 4–6 characters	**DONE** **12 characters**
5	**Auth + coin-op**	Privy login, profiles, deposits to credit ledger, ranked queue	**DONE** **AIR; no deposits**
6	**Wagering**	Escrow contract + verified settlement; gated on legal review	**PARTIAL** **off-chain only**

Read that table twice. Six of seven rows are green or amber on a spec that estimated this as a multi-month v1. The single red row is the one the spec said was load-bearing for all the others.

Architecture as built

The spec's shape — one deterministic core, three consumers — survived contact with reality intact. This is the report's most reassuring finding: the structure did not rot under speed.

Package	Source	Role	Verdict
`@af/core`	2,952	Deterministic sim. Zero dependencies, zero DOM. The crown jewel.	**Healthy**
`@af/client`	6,376	Canvas 2D renderer, input, netcode, UI, FX, audio, touch.	**Accreting**
`@af/server`	1,957	Matchmaking, input ledger, re-sim verification, economy, AIR.	**Healthy**
`@af/studio`	4,586	Character authoring + AI sprite pipeline. Local only, never deployed.	**Healthy**

@af/core is dependency-free and stayed that way. The client's only runtime dependency is the AIR Kit SDK. There is no build-tool sprawl, no framework lock-in, and the entire test suite runs on `tsx --test` with no framework at all. For a codebase assembled this fast, that restraint is the reason the rest of this audit is short.

CRITICAL

Red flags

Ordered by consequence, not by how easy they are to fix.

RED FLAG 1 — The M1 feel gate has never been run. This is the top risk in the project.

The spec is unusually blunt: *"Milestones 0–1 are the whole ballgame: if the core doesn't feel like MvC with one character, nothing downstream matters."* Five milestones — Studio, 12 characters, netcode, economy, wagering — are now stacked on a foundation whose stated acceptance test has not been attempted. Zero external playtests exist.

Why this compounds: the tuning knobs (`TUNING` in `core/src/data.ts`, plus per-character frame data) get harder to move as the roster grows. Retuning against 1 character was cheap. Against 12 characters, each with 98 authored sprite steps and pinned golden hashes, it is a project. Every hour spent downstream is leveraged on an unvalidated assumption, and the leverage is still increasing.

RED FLAG 2 — No cross-environment determinism test. The money path depends on it.

Spec §2.1 requires the CI gate assert bit-identical hashes *"cross-environment (browser vs Node)"*. What exists is Node-only: golden replay hashes, mechanical guards, serialize and snapshot completeness. All excellent — all one environment.

The entire settlement path assumes a browser sim and a Node re-sim agree bit-for-bit. If they ever drift, **honest players get flagged as deviators and forfeit real credits.** This has been proven empirically (live matches verified, a 4,918-tick match bit-identical, no deviator) but it is **not gated**. Empirical evidence expires the next time someone touches sim arithmetic — and someone is touching it right now (see red flag 3).

RED FLAG 3 — Concurrent sessions are editing the deterministic core live.

During this audit `verify` went red with `Cannot find name 'SUPER_SKILL_FLOOR'` at a line number that did not match the file on disk. `core/src/ai.ts` had been written 23 seconds earlier. Re-running was green. That was a race with another live writer, not a breakage — but it means the working tree was not a stable audit target.

Seven files are uncommitted, including a new `client/src/autospecial.ts`. Two agents editing `@af/core` concurrently is precisely how a golden-hash conflict or a lost fix happens — and a golden-hash conflict *looks exactly like a determinism bug*, which is the most expensive class of false alarm this project can generate.

RED FLAG 4 — The economy is live with an open farming surface.

Real credits, real Supabase, real settlement, in production. The sybil and rate-limit knobs flagged in ADR 0004 remain open: nothing observed prevents N accounts each claiming the daily +10 grant. Solo-farm caps are unimplemented.

This is tolerable today for exactly one reason: credits cannot cash out. That property is now load-bearing and undocumented. It should be written down as an explicit release gate, because the moment anyone implements cash-out without closing these knobs, the farming surface becomes a financial one — and spec §9 Phase C's legal-review gate arrives with it.

RED FLAG 5 — Money logic lives twice, and only one copy can be unit-tested.

Settlement exists in `supabase/migrations/*.sql` and is mirrored in `server/src/persist.ts`. The project has already been bitten here: `NULL = s` is NULL, not false, so a null deviator poisoned the won/lost booleans and settled **both sides as losers** on real Postgres. The TS mirror cannot catch SQL semantics — by construction.

The bug was fixed. The *class* was not. Every future money change needs a real-Postgres smoke test; a green suite proves nothing about the copy that actually moves credits.

RED FLAG 6 — Deferred structural debt, now accruing interest.

Entity model is not tag-ready. Spec §2.2 said do not hardcode P1/P2; the code is `fighters: [FighterState, FighterState]` (`state.ts:103`), baked into snapshot/restore/serialize and the golden hash format. Discussed as an open decision below.

Client god-files. `ui.ts` is 1,951 lines and `main.ts` is 1,190 — together 49% of the client. The VS card, wallet strip, and stakes overlay all landed here.

VS poses authored for 1 of 12 characters. Analog only; the other 11 fall back to select portraits — safe, but inconsistent, which is the exact bug the VS-slot system was built to fix.

WebRTC P2P is deferred, and the relay is currently a security feature. Peers never learn each other's IPs, so DDoS-the-opponent is moot. ADR 0005 already flags that this dies with the P2P upgrade.

DIVERGENCE

Spec deviations — and whether to defend them

Each of these is a real divergence from the written spec. Most are defensible. They are listed so the choice becomes explicit rather than ambient — an undocumented deviation is an invitation for a future agent to "fix" working code back toward a stale document.

Spec said	We built	Assessment	Action
PixiJS v8, WebGL2 (§6)	Canvas 2D, zero deps	Measured 1.9 ms/frame against a 16.7 ms budget; demo bundle 332 KB against a 5 MB ceiling. The perf case for Pixi never materialised. Declared openly in package.json.	**Amend spec**
WebRTC P2P DataChannel (§7.1)	WebSocket relay	Rollback, prediction and resim are all real and transport-agnostic (net.ts:9). Spec treats relay as the fallback. The relay is also what hides peer IPs today.	**Amend spec**
Privy auth (§9)	AIR Kit (Moca)	Arguably an upgrade — the AIR account doubles as the wallet. §9's real obligation, that identity flows through a provider from the start, is met.	**Amend spec**
zod validation (§3)	Hand-rolled validation	loadCharacter() throws on duplicate ids, empty steps, non-integer frames, missing buttons/motions/meterCost (data.ts:158+). Equivalent for fields covered; keeps core dependency-free.	**Accept**
Snapshot ring ~10 (§7.1)	128 (net.ts:25)	Larger is safer. The ring-wrap hazard is understood and bounded.	**Accept**
Charge moves (§4)	Not implemented	No character needs them. A coverage gap, not a defect.	**Backlog**
On-chain deposit to ledger (§9 Phase B)	Daily +10 grant	"Insert coin" mints rather than deposits. No cash-out — which is what keeps this out of regulatory scope, and also what makes it not yet the spec's coin-op.	**Decide**
Clients + server sign the input-log hash (§9 Phase C)	Not implemented	Signing infrastructure exists (air-issuer.ts, airjwt.ts) but points at credentials, not ledgers. Correctly not-yet-due — but a hard prerequisite for real wagering.	**Blocker for M6**
Entity pool, teams own fighters (§2.2)	Hardcoded 2-tuple	The one deviation the spec argued against *in advance*. See the open decision overleaf.	**Decide**

CREDIT WHERE DUE

What is genuinely strong

An audit that lists only problems misleads. These are the things this build got right that most projects at this velocity get wrong, and they are the reason the red-flag list above is as short as it is.

Determinism discipline exceeds what the spec asked for

`guards.test.ts` mechanically bans the entire hazard class — `Math.random`, `Date`, DOM, timers, `fetch`, `process`, transcendental math — rather than trusting code review to catch it. `serialize.test.ts` proves *every* scalar field reaches the hash by mutating each one and asserting the hash moves. Golden hashes pin six full matches, and have already caught a real behaviour change (a 900-to-899 damage-scaling tweak). This is the part most projects fake, and this one didn't.

The InputSource abstraction paid off, visibly

The uncommitted `autospecial.ts` is the proof. It delivers tap-to-special by *synthesising the exact buttons a human would press* and feeding them through the normal `InputFrame` path. It therefore cannot desync, works

under rollback and online for free, and required zero core changes. Its header comment reasons correctly from the server-reverify constraint to the design. That is the spec's §2.2 abstraction being honoured under deadline pressure — the moment most architectures get shortcut.

The settlement ladder is real engineering

ADR 0005's "the ledger is the truth" makes winning-then-disconnecting safe and ragequitting unprofitable — the non-obvious right answer. It fixed two live money bugs: a win-then-drop was booking the winner as forfeit-loser, and a both-disconnect raced two timers so the first socket to close arbitrarily lost real credits. The idle-forfeit sweep closes the lag-switch hole where holding a socket open froze an opponent's pot as a free option.

Operational hygiene

- **Secrets are clean.** `.env`, `air/partner_rs256.pem`, vendored UMD, and tool binaries are all gitignored; only `.env.example` and the keygen script are tracked. Nothing sensitive has ever been committed.
- **Zero TODO, FIXME, or HACK markers** across all four packages' source.
- **Five ADRs** record the load-bearing decisions, and the hard-won lessons are written down where the next agent will hit them rather than lost in chat history.
- **M2's exit test didn't just pass, it embarrassed the budget:** under 2 days allowed, ~5 minutes actual.

RECOMMENDED NEXT STEPS

What to do, and why

In priority order. Each carries its defence, because a recommendation without a reason is just a preference.

P0 Run the M1 feel gate this week, before anything else ships.

Do: Put the Vercel build in front of two or three people who actually play Marvel vs Capcom. Ask them one question: does it feel right? Capture the answer. If it fails, stop feature work and tune `TUNING` plus Analog's frame data until it passes.

Why: This is the spec's own blocking gate and the only item on this list that can invalidate the other five milestones. It is now practical solo — VS AGENT works, the loop is polished with quick match and instant rematch, and prod is live. It will never be cheaper than it is today: every character added, every golden hash pinned, and every match settled against current frame data raises the cost of acting on a bad answer. Three days of building have been an unhedged bet on this gate passing. Settle the bet.

P1 Land a cross-environment determinism gate in CI.

Do: Run the existing golden scenarios in a headless browser and assert the resulting hashes match the Node ones. The scenarios already exist in `core/test/golden/`; this is wiring, not new test design.

Why: It is the one spec-mandated gate that is missing, and it is the gate the money path silently depends on. Today the browser-Node equivalence is an empirical observation from live matches, not an enforced invariant — and the core is under active concurrent edit. Without this gate, the first float leak does not surface as a red build; it surfaces as a support ticket from a player who lost real credits to a desync they did not cause. That is the single worst failure mode this product has, and it is currently undefended.

P2 Serialise the concurrent-session problem.

Do: Commit or shelve the seven in-flight files. Then establish that exactly one session owns `@af/core` at a time — the AGENTS.md contract is the right place to say so.

Why: Two agents editing the deterministic core concurrently will eventually produce a golden-hash conflict. That conflict will look exactly like a determinism bug, and it will be debugged as one — burning hours on the most expensive false alarm the project can generate. I hit a live instance of this race during the audit itself, which is how it got onto the list.

P3 Close the economy knobs before any cash-out conversation.

Do: Implement the sybil rate limits and solo-farm cap already named in ADR 0004. Write down "credits cannot cash out" as an explicit release gate rather than an implementation detail.

Why: The farming surface is open and the economy is live. Right now that is safe for exactly one reason — credits have no exit — and that reason is nowhere in writing. The failure mode is not that someone farms credits; it is that someone implements cash-out in six weeks, reasonably assuming the economy was built to be exit-safe, and mints a financial liability on the way. Spec §9 Phase C's legal-review gate arrives at the same moment. Close the knobs while they are still cheap and theoretical.

P4 Amend the spec to match reality.

Do: Update Build Spec v1: Canvas 2D over PixiJS, WS relay over WebRTC P2P, AIR Kit over Privy, hand-rolled validation over zod. One ADR, or four short edits.

Why: Cheapest task on this list and the highest ratio of harm prevented to effort spent. This project is explicitly built for cross-model handoff — the user switches to Grok in Cursor when Claude limits hit, and AGENTS.md exists precisely to carry context across that boundary. A stale spec is an active instruction to a future agent to "fix" working, deliberate code back toward a document nobody meant to keep. All four deviations are defensible; leaving them undocumented is not.

P5 Housekeeping, in the gaps.

Do: Author VS poses for the remaining 11 characters via Studio. Split `ui.ts` and `main.ts` when they next resist a change. Add the startup smoke test for money-path SQL against real Postgres.

Why: None of this is urgent and none of it should displace P0. But the VS-pose gap is a visible inconsistency in the exact system built to fix it, and the two client god-files are 49% of the client — the next UI feature will pay for the deferral. Do these when they stop being optional.

DECIDE NOW

The open question: is tag/assist ever happening?

Spec §2.2 is emphatic, and worth quoting because it anticipated this exact moment:

> **Build Spec v1, §2.2**
>
> *"Do not hardcode 'player 1 fighter' and 'player 2 fighter'. The sim owns a pool of entities (fighters, projectiles, effects), and each team owns a list of fighter entities with one flagged active. In v1 each team's list has length 1. Tag/assists later = spawn a second fighter entity per team + a swap/assist-call mechanic — no engine rewrite."*

What exists is `fighters: [FighterState, FighterState]` (`core/src/state.ts:103`) — a hardcoded two-tuple, threaded through `snapshot`, `restore`, `serialize`, the golden hashes, and the server's re-simulation.

This is the one deviation the spec argued against *in advance*, and the reason it gave — retrofitting is a rewrite, building on it is cheap — still holds. The cost has only gone up: the tuple is now baked into the hash format that the client, the server, the golden tests, and every settled match in Supabase all agree on. Changing it is not a refactor of one file; it is a protocol migration with money already denominated in the old format.

So decide deliberately rather than by drift. There are three possible states and the project is currently in the worst one:

State	What it costs	Verdict
Write tag off. An ADR says v1 and v2 are 1v1; the 2-tuple is the intended design.	Nothing. The 2-tuple is simpler, it works, and it is shipping. Reclaims the spec's honesty at the cost of a feature nobody has asked for.	**Cheapest**
Do the refactor now. Entity pool + teams, migrate the hash format, re-bless goldens.	Expensive, and it touches the one module that must not break. But it only gets worse: every new character, golden hash, and settled match raises the price.	**Defensible**
Leave it ambient. Spec claims tag-ready; code isn't; nobody has decided.	The worst of both. Pays the refactor's future cost *and* misleads every agent who reads the spec as truth — in a project explicitly designed for cross-model handoff.	**Current state**

Recommendation: write it off. Nothing in three days of building has needed tag, the roster is 12 characters of 1v1 identity, and the economy is denominated in 1v1 outcomes. If tag ever becomes real, it will be real enough to fund its own migration — and it will arrive with a reason, which the current architecture-on-spec does not have. Take the honest simpler design and say so in an ADR.

APPENDIX

Verification evidence

Everything asserted in this report was checked against the running system on 17 July 2026, not inferred from documentation.

Check	Result
`npm run verify`	GREEN. Typecheck across @af/core, @af/client, @af/studio, @af/server; 67 core tests; 28 server tests; 0 failures. (One transient red from a concurrent writer — see red flag 3.)
Match server health	`GET match-server-production.up.railway.app/` returns `{engine:"af-core-1", protocol:3, persistence:true, characters:[12], stages:[3]}`
Client	`agent-fighter.vercel.app` returns HTTP 200

Check	Result
Secrets audit	`git ls-files` across env/pem/key patterns returns only `.env.example` and `tools/air-keygen.mjs`. Clean.
Dead code markers	No TODO / FIXME / HACK / XXX in any package source.
Determinism guards	guards.test.ts bans Math.random, Date, DOM, globalThis, rAF, timers, fetch, storage, process in @af/core.
Golden lock	`core/test/golden/hashes.json` pins 6 full-match hashes.
Bundle size	Demo bundle 332 KB against the spec's 5 MB core ceiling.
Renderer perf	1.9 ms per frame measured against a 16.7 ms budget (perf overlay, F key in game).
Working tree	7 files uncommitted at audit time, incl. new `client/src/autospecial.ts`. Left untouched — another session held the core.

Scope and method. This audit walked Build Spec v1 section by section against the working tree, ran the verify gate, checked production endpoints, and reconstructed the timeline from 67 commits rather than from project notes. Three parallel audit subagents were dispatched and all three died on a session limit before returning findings, so coverage is targeted rather than exhaustive: the core, server, economy SQL, netcode transport, entity model, secrets, and determinism gates were examined directly; the Studio pipeline and client renderer were sampled. A deeper pass on packages/studio and the sprite QC thresholds remains available if wanted.